

CSE 144 Final Project Report

Team Information

- **Team members:** Samarth Ramesh, Virak Chum
- **Kaggle team / submission name:** SamarthRameshVirakChum
- **Repository:** <https://github.com/sshramesh/cse144-final-project>
- **Final submitted CSV:** `submissions/SamarthRameshVirakChumKaggleSubmission_1036.csv`
- **Public Kaggle score:** 85.454

1. Project Overview

This project addresses the UCSC CSE 144 transfer-learning image classification challenge. The task is to train a classifier using the labeled images in `train/` and produce labels for the unlabeled images in `test/`. The final prediction file must be a CSV with two columns:

```
ID, Label
```

The assignment requires a public GitHub repository containing source code, a PDF report, a link to model weights, and a leaderboard screenshot. It also emphasizes reproducibility: the submitted code and documentation should allow another person to reproduce the final training and inference results.

2. Dataset

The competition dataset contains 100 image classes. The training data is organized as class folders named 0 through 99, and each folder contains images for the corresponding label. The assignment instructions warn that label order is critical: folder 0 must map to label 0, folder 1 must map to label 1, and so on.

The local dataset used for development contained:

Split	Count	Notes
Training images	1079	100 class folders
Classes	100	Numeric labels 0 through 99
Images per class	4 to 41	The local distribution was not perfectly uniform
Template submission IDs	1000	From <code>sample_submission.csv</code>
Actual test images	1036	All images under <code>test/</code>

Although the assignment handout and sample submission describe 1000 test images, Kaggle expected 1036 submission rows for this competition instance. The final pipeline therefore uses all images in `test/`, sorted by numeric filename stem, instead of relying only on `sample_submission.csv`.

3. Method

Because the dataset is small relative to the number of classes, training a large image model from scratch would be unreliable and likely to overfit. The final approach uses transfer learning through pretrained image embeddings:

1. Load strong pretrained image encoders from `timm`.
2. Apply each model's official evaluation transform.
3. Extract one normalized feature vector per image.

4. Apply manually tuned weights to the feature blocks.
5. Concatenate and L2-normalize the combined features.
6. Train a balanced logistic regression classifier.
7. Predict labels for all test images and write a Kaggle submission CSV.

The final submitted model used three pretrained backbones:

Backbone	Role
convnext_base.fb_in22k_ft_in1k_384	ConvNeXt feature extractor pretrained on ImageNet-22K and fine-tuned on ImageNet-1K
swin_base_patch4_window12_384.ms_in22k_ft_in1k	Swin Transformer feature extractor pretrained on ImageNet-22K and fine-tuned on ImageNet-1K
vit_base_patch14_reg4_dinov2.lvd142m	DINOv2 ViT feature extractor trained with self-supervised learning

The final classifier was:

Balanced logistic regression, C = 200

The final feature block weights were:

ConvNeXt: 0.25
Swin: 0.75
DINOv2: 1.50

The codebase also supports prototype classification and cosine k-nearest neighbors, but the best local-validation result came from using logistic regression alone. In the submitted run, the probability blend weights were:

logistic regression: 1.0
prototype classifier: 0.0
k-nearest neighbors: 0.0

4. Implementation Details

The main submission pipeline is implemented in:

src/cse144_project/make_embedding_submission.py

Important implementation choices:

- **Numeric labels:** `read_train_samples` parses class folder names as integers, preventing alphabetical label-order mistakes.
- **Image preprocessing:** transforms are created through `timm.data.resolve_data_config` and `timm.data.create_transform`, so each pretrained model receives the image size and normalization it expects.
- **Feature normalization:** individual model outputs are normalized, then weighted feature blocks are concatenated and normalized again.
- **Cross-validation:** hyperparameters are evaluated with stratified 4-fold cross-validation.
- **Class imbalance handling:** logistic regression uses `class_weight="balanced"`.
- **Caching:** extracted features are cached under `artifacts/features/` so repeated runs do not need to re-run all pretrained backbones.
- **Device selection:** the code selects CUDA first, then Apple MPS, then CPU.
- **Fixed seed:** the pipeline uses seed 144.

The exact reproduction script is:

```
scripts/run_best_85_submission.sh
```

Its core command is:

```
python -m cse144_project.make_embedding_submission \
--data-dir "$DATA_DIR" \
--all-test \
--models \
  convnext_base.fb_in22k_ft_in1k_384 \
  swin_base_patch4_window12_384.ms_in22k_ft_in1k \
  vit_base_patch14_reg4_dinov2.lvd142m \
--feature-block-weights 0.25,0.75,1.5 \
--blend-weights 1,0,0 \
--c-grid 200 \
--knn-grid 5 \
--proto-temperature 6 \
--no-tta-flip \
--batch-size 16 \
--workers 2 \
--seed 144 \
--output submissions/SamarthRameshVirakChumKaggleSubmission_1036.csv
```

5. Experiments

Several pretrained embedding configurations were tested. The table below reports local stratified cross-validation accuracy. The public Kaggle score is only listed for the final submitted CSV.

Run	Method	Local CV Accuracy	Kaggle Public Score	Notes
1	ConvNeXt Base embeddings	0.7692 +/- 0.0087	Not submitted	Quick one-backbone baseline
2	ConvNeXt Base + Swin Base	0.7905 +/- 0.0107	Not submitted	First two-backbone ensemble
3	Tuned ConvNeXt Base + Swin Base	0.8035 +/- 0.0100	Not submitted	Improved regularization and feature settings
4	ConvNeXt Base + Swin Base + DINOv2 Base	0.8656 +/- 0.0122	Not submitted	Large improvement after adding DINOv2
5	Block-weighted ConvNeXt + Swin + DINOv2	0.8767 +/- 0.0151	85.454	Final submitted model

The largest improvement came from adding DINOv2 embeddings, suggesting that self-supervised pretrained features transferred well to this diverse 100-class dataset. The final block weighting further improved validation accuracy by reducing the influence of ConvNeXt and increasing the influence of DINOv2.

6. Final Result

The final Kaggle submission file is:

```
submissions/SamarthRameshVirakChumKaggleSubmission_1036.csv
```

The generated file has:

```
1036 rows
2 columns: ID, Label
labels from 0 through 99
100 unique predicted labels
```

The public Kaggle score reported by the leaderboard was:

```
85.454
```

The assignment states that the public leaderboard reflects only an estimated subset of the test data, so the public score was used mainly to verify that the format was accepted and that the model was performing well above the 60% baseline.

7. Reproducibility Instructions

From a fresh clone of the repository:

```
git clone https://github.com/sshramesh/cse144-final-project
cd cse144-final-project
```

Create the Python environment:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
pip install -e .
```

Run the submitted pipeline:

```
./scripts/run_best_85_submission.sh /path/to/ucsc-cse-144-spring-2026-final-project
```

Verify the output file:

```
python - <<'PY'
import pandas as pd

path = "submissions/SamarthRameshVirakChumKaggleSubmission_1036.csv"
df = pd.read_csv(path)
print(df.shape)
print(df.columns.tolist())
print(df["Label"].min(), df["Label"].max(), df["Label"].nunique())
PY
```

Expected output:

```
(1036, 2)
['ID', 'Label']
0 99 100
```

The first run may take time because it downloads pretrained weights and extracts embeddings from three backbones. Later runs are faster because feature caches are stored under `artifacts/features/`.

8. Required Repository Artifacts

The assignment requires the public GitHub repository to contain or reference the following:

Requirement	Location / Status
Source code	<code>src/cse144_project/</code>
Training / inference instructions	<code>README.md</code> and this report
Final submission CSV	<code>submissions/SamarthRameshVirakChumKaggleSubmission_1036.csv</code>
PDF report	<code>report/Project_Report.pdf</code>
Model weights link	<code>weights/README.md</code>
Kaggle leaderboard screenshot	<code>assets/kaggle_leaderboard.png</code>

The model used for the final submission is based on public pretrained weights downloaded through `timm` plus a lightweight logistic regression classifier trained from cached embeddings. The feature caches and downloaded pretrained weights are intentionally not committed to git because they are generated artifacts.

9. Limitations and Future Work

This approach performs well for the assignment because it uses strong pretrained representations and avoids overfitting a large neural network on a small dataset. However, there are several possible improvements:

- Train a calibrated ensemble over multiple cross-validation folds.
- Add test-time augmentation if runtime allows.
- Fine-tune one or more pretrained backbones with careful regularization.
- Explore stronger pretrained models while monitoring overfitting and runtime.
- Build a more systematic hyperparameter search for feature block weights and classifier settings.

10. Grading Context

The assignment score is based on three parts:

1. Kaggle test accuracy, with a 60% baseline.
2. Presentation score.
3. Report, code, and model-weight submission quality.

With a public Kaggle score of 85.454, the final submitted model is well above the 60% baseline. The repository and report are organized to support the remaining reproducibility and documentation requirements.